МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка

ЗВІТ

до лабораторної роботи №3
з дисципліни «Професійний Python»

Тема: «Ітератори, генератори та потокова обробка даних»

Виконав:
студент Левандівський Назар

Перевірив:
Куньо Іван Михайлович

Львів – 2026

Мета роботи

Сформувати практичні навички роботи з ітераційною моделлю Python: iterable та iterator, власні iterator classes, generator functions, yield, yield from, generator expressions, модуль itertools, lazy evaluation та потокова обробка великих наборів даних із порівнянням eager і lazy підходів за часом і пам'яттю.

1. Постановка задачі

Варіант 1. Потокова обробка даних системи обліку студентів (узгоджено з ЛР1–ЛР2). Вхід – CSV-журнал студентів `student_id,first_name,last_name,group,average_grade`. Потрібно побудувати lazy-конвеєр source → reading → cleaning → parsing → validation → filtering → transformation → aggregation/batching, обробити не менше 100 000 записів, порівняти eager і lazy варіанти та оцінити пам'ять.

Таблиця 1 – Вимоги варіанта 1 та їх реалізація (результат на data/input.csv)

Вимога	**Реалізація**	**Результат**
streaming CSV reading та validation студентів	`read_lines`, `validate_students`	відсіяно 6 рядків із 25
фільтрація за групою та мін. середнім балом	`filter_by_group`, `filter_by_min_grade`	PI-22: 5 студентів; ≥ 90: 6 студентів
пошук за прізвищем або ID (lazy)	`find_by_last_name`, `find_by_id`	ID 3 знайдено після 11 рядків; «мельник» – 3 студенти
streaming average середнього бала	`streaming_average`, `summarize`	80.91
min/max без повної матеріалізації	`min_max_grade`	54.30 (Роман Лисак) / 98.40 (Ігор Романюк)
перші N студентів із найвищим балом за умовою	`top_students` (heapq), `first_n_students` (islice)	Ігор Романюк, Оксана Мороз, Ірина Бондар
batch processing записів	`Batcher`, `batched_by_group`, `batch_statistics`	4 пакети по ≤ 5; 7 динамічних за групою
підрахунок студентів за групами	`count_by_group`, `summarize`	PI-21 – 6, PI-22 – 5, PI-23 – 5, PI-24 – 3

2. Вхідні дані

Кожен рядок журналу – один студент. Правила validation: `student_id` – додатне ціле, ім'я та прізвище не порожні, `group` має формат `PI-21` (дві літери, дефіс, дві цифри), `average_grade` – скінченне число від 0 до 100. Причина кожного відхилення рахується в `Counter`: `bad_id`, `empty_name`, `bad_group`, `bad_grade`, `grade_out_of_range`.

Демонстраційний файл `data/input.csv` містить коментар, порожній рядок і 25 записів, відсортованих за групами; серед них 6 навмисно некоректних. Для експерименту нескінченний генератор `random_students` створює журнал (≈1 % зіпсованих рядків), а `islice` обмежує його до 10 000 …

1 000 000 записів; `data/journal/` – той самий потік, розрізаний на 3 файли в підкаталогах.

3. Структура проєкту

```
Lab_03/
├── pyproject.toml, README.md, .gitignore
├── data/
│   ├── input.csv           демонстраційний журнал студентів
│   ├── journal/            журнал із 3 файлів у підкаталогах
│   └── generated/          набори для експерименту (не в git)
├── src/stream_processor/   __init__.py, analytics.py, batches.py, datagen.py
│                           experiment.py, filters.py, main.py, models.py
│                           parsers.py, pipeline.py, readers.py, transformations.py
├── tests/                  test_analytics.py, test_stages.py
├── results/                experiment.csv, early_termination.csv, …
└── report/build_report.py  генератор цього звіту
```

4. Iterable та iterator

Iterable – об'єкт, з якого `iter()` отримує iterator; iterator зберігає стан обходу, віддає елементи через `__next__()` і завершується `StopIteration`. Цикл `for` викликає `iter()`, потім `next()` до `StopIteration`. Власний iterator `Batcher` віддає пакети студентів; він one-shot (`iter(b) is b`). `JournalSource` – окремий iterable: кожен `iter()` створює новий генератор файлів, тому його можна обходити повторно.

```
class Batcher(Generic[T]):
    def __init__(self, items: Iterable[T], size: int) -> None:
        if size < 1:
            raise ValueError("Розмір пакета має бути >= 1")
        self._iterator = iter(items)
        self._size = size

    def __iter__(self) -> "Batcher[T]":
        return self

    def __next__(self) -> list[T]:
        batch = list(islice(self._iterator, self._size))
        if not batch:
            raise StopIteration
        return batch

class JournalSource:
    def __init__(self, root: Path) -> None:
        self.root = root

    def __iter__(self) -> Iterator[Path]:
        return iter_csv_files(self.root)
```

5. Генератори, yield і yield from

Streaming reader і рекурсивний пошук файлів використовують `yield from`; файл відкритий, доки генератор не вичерпано або не закрито:

```
def read_lines(path: Path) -> Iterator[str]:
    with path.open("r", encoding="utf-8-sig", newline="") as file:
        yield from file

def iter_csv_files(root: Path) -> Iterator[Path]:
    for entry in sorted(root.iterdir()):
        if entry.is_dir():
            yield from iter_csv_files(entry)
        elif entry.suffix == ".csv":
            yield entry
```

Transformation stage – generator function з `yield`: кожен студент отримує поле `level` (рівень успішності) лише тоді, коли його запитує наступний етап. Етап не має власного стану, тому пам'ять не залежить від кількості студентів:

```
def grade_level(grade: float) -> str:
    return next(label for threshold, label in GRADE_LEVELS if grade >= threshold)

def enrich_students(students: Iterable[Student]) -> Iterator[StudentRecord]:
    for student in students:
        yield StudentRecord(*student, level=grade_level(student.average_grade))
```

Generator expressions використано у `build_pipeline` (`read_rows(path) for path in files`), `find_first`, `first_n_students`, `top_students` і `running_average`.

Таблиця 2 – Використані засоби itertools

Засіб	Де використано	Призначення
`islice`	`Batcher`, `first_n_students`, datagen	пакети, перші N, обмеження нескінченного потоку
`chain.from_iterable`	`build_pipeline`	один потік із кількох файлів
`accumulate`	`running_average`	накопичене середнє після кожного студента
`groupby`	`batched_by_group`, `batch_statistics`	пакети за групою; статистика пакета (після sorted)
`count`	`random_students`	ідентифікатори студентів
`takewhile`	`main`	студенти до першого з балом < 60

6. Lazy pipeline

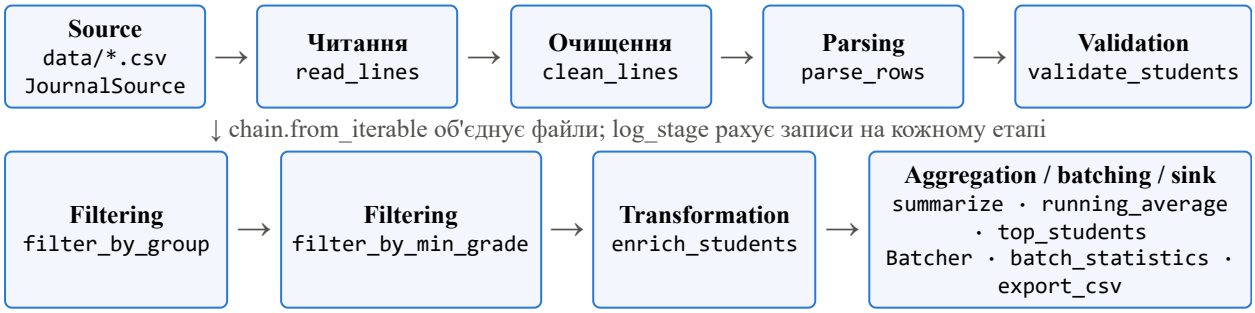


Рисунок 1 – Схема потокового конвеєра

Функція `build_pipeline` лише з'єднує генератори – жоден рядок не читається, доки результат не почне споживати sink:

```
def build_pipeline(
    paths: Path | Iterable[Path],
    *,
    group: str | None = None,
    min_grade: float | None = None,
    errors: Counter[str] | None = None,
    stages: Counter[str] | None = None,
) -> Iterator[StudentRecord]:
    files = [paths] if isinstance(paths, Path) else paths
    rows = chain.from_iterable(read_rows(path, stages) for path in files)
    students = log_stage("4 validate", validate_students(rows, errors), stages)
    if group is not None:
        students = log_stage("5 filter group", filter_by_group(students, group), stages)
    if min_grade is not None:
        students = log_stage("6 filter grade", filter_by_min_grade(students, min_grade),
stages)
    return iter(log_stage("7 transform", enrich_students(students), stages))
```

Validation, filtering, топ-N, streaming aggregation і динамічні пакети. `nlargest` тримає в пам'яті лише N кандидатів, а не весь потік:

```
def validate_students(
    rows: Iterable[dict[str, str]],
    errors: Counter[str] | None = None,
) -> Iterator[Student]:
    for row in rows:
        result = parse_student(row)
        if isinstance(result, str):
            if errors is not None:
                errors[result] += 1
            continue
        yield result

def filter_by_min_grade(students: Iterable[Student], minimum: float) -> Iterator[Student]:
    for student in students:
        if student.average_grade >= minimum:
            yield student

def top_students(
    records: Iterable[StudentRecord],
    n: int,
    predicate: Callable[[StudentRecord], bool] | None = None,
) -> list[StudentRecord]:
    candidates = records if predicate is None else (r for r in records if predicate(r))
    return nlargest(n, candidates, key=attrgetter("average_grade"))

def streaming_average(values: Iterable[float]) -> float:
    total = 0.0
    count = 0
    for value in values:
        total += value
        count += 1
    if count == 0:
        return 0.0
    return round(total / count, 2)

def running_average(records: Iterable[StudentRecord]) -> Iterator[float]:
    grades = (record.average_grade for record in records)
    return (round(total / n, 2) for n, total in enumerate(accumulate(grades), 1))

def batched_by_group(
    records: Iterable[StudentRecord],
    max_size: int = 100,
) -> Iterator[list[StudentRecord]]:
    for _, members in groupby(records, key=attrgetter("group")):
        yield from Batcher(members, max_size)
```

7. Результат роботи програми

Вивід `python -m stream_processor.main`:

```
ЛР3, варіант 1: потокова обробка даних системи обліку студентів

Етапи конвеєра (скільки записів пройшло):
  1 read=28, 2 clean=26, 3 parse=25, 4 validate=19, 7 transform=19
Відхилено під час validation: 6 {'bad_id': 1, 'empty_name': 1, 'bad_group': 2, 'bad_grade': 1,
'grade_out_of_range': 1}

Оброблені студенти (input.csv):
    1 Анна Коваль            PI-21   91.50  відмінно
    2 Олег Мельник           PI-21   84.00  добре
    4 Андрій Шевченко        PI-21   88.70  добре
   10 Максим Мельник         PI-21   70.60  задовільно
   14 Юлія Кравець           PI-21   89.90  добре
   17 Роман Лисак            PI-21   54.30  незадовільно
    3 Ірина Бондар           PI-22   95.20  відмінно
    5 Марія Бойко            PI-22   79.30  добре
    7 Софія Гнатюк           PI-22   66.80  задовільно
   13 Назар Савчук           PI-22   61.20  задовільно
   19 Ігор Романюк           PI-22   98.40  відмінно
    6 Тарас Гриценко         PI-23   72.40  задовільно
    8 Дмитро Ткаченко        PI-23   58.90  незадовільно
    9 Катерина Лисенко       PI-23   93.10  відмінно
   15 Богдан Ткачук          PI-23   77.50  добре
   18 Вікторія Бондаренко    PI-23   83.10  добре
   11 Василь Петренко        PI-24   85.40  добре
   12 Оксана Мороз           PI-24   97.00  відмінно
   16 Лілія Мельник          PI-24   90.00  відмінно

Фільтр за групою PI-22:
    3 Ірина Бондар           PI-22   95.20  відмінно
    5 Марія Бойко            PI-22   79.30  добре
    7 Софія Гнатюк           PI-22   66.80  задовільно
   13 Назар Савчук           PI-22   61.20  задовільно
   19 Ігор Романюк           PI-22   98.40  відмінно

Фільтр average_grade >= 90: [1, 3, 19, 9, 12, 16]
Група PI-21 і average_grade >= 85: ['Анна Коваль', 'Андрій Шевченко', 'Юлія Кравець']

Пошук за ID 3: Ірина Бондар, PI-22; прочитано рядків: 11 (early termination)
Пошук за прізвищем 'мельник' (lazy generator): ['Олег Мельник', 'Максим Мельник', 'Лілія Мельник']
  перший збіг Олег Мельник знайдено після 4 рядків

Агрегація (summarize, один прохід):
  студентів 19, середній бал 80.91
  мінімум 54.30, максимум 98.40, найкращий: Ігор Романюк
  за групами: {'PI-21': 6, 'PI-22': 5, 'PI-23': 5, 'PI-24': 3}
  середній бал за групами: {'PI-21': 79.83, 'PI-22': 80.18, 'PI-23': 77.0, 'PI-24': 90.8}
streaming_average без list: 80.91
min_max_grade: мінімум Роман Лисак 54.3,
               максимум Ігор Романюк 98.4
count_by_group (Counter): {'PI-21': 6, 'PI-22': 5, 'PI-23': 5, 'PI-24': 3}
Накопичене середнє (accumulate): [91.5, 87.75, 88.07] ... 80.91
takewhile: студентів до першого з балом < 60 - 5

Топ-3 за балом (heapq.nlargest, пам'ять O(N)): ['Ігор Романюк', 'Оксана Мороз', 'Ірина Бондар']
Топ-2 у групі PI-21: ['Анна Коваль', 'Юлія Кравець']
Перші 4 студенти з балом >= 85 (islice): [1, 4, 14, 3]

Batch processing (Batcher, size=5) і статистика пакета (groupby):
  пакет 1: студентів 5, {'PI-21': (5, 84.94)}
  пакет 2: студентів 5, {'PI-21': (1, 54.3), 'PI-22': (4, 75.62)}
  пакет 3: студентів 5, {'PI-22': (1, 98.4), 'PI-23': (4, 75.47)}
  пакет 4: студентів 4, {'PI-23': (1, 83.1), 'PI-24': (3, 90.8)}
Динамічні пакети за групою (max_size=4): розміри [4, 2, 4, 1, 4, 1, 3]

Журнал із кількох файлів (рекурсивний yield from): 3 файли, повторний обхід дає 3
  chain: 693 студентів, середній бал 69.83, некоректних рядків 7
  за групами: {'PI-21': 136, 'PI-22': 160, 'PI-23': 134, 'PI-24': 120, 'PI-25': 143}
```

```
Нескінченний генератор + islice(..., 2):
  1,Ірина,Ткаченко,PI-21,55.30
  2,Катерина,Гриценко,PI-24,87.32
Streaming export відмінників: 6 -> results\excellent_students.csv
```

Ручна перевірка: суми балів за групами 479.0 + 400.9 + 385.0 + 272.4 = 1 537.3; 1 537.3 / 19 = 80.91, що збігається з `summarize` і `streaming_average`; жоден бал не виходить за межі 0–100.

8. Eager і lazy: експеримент

Eager-версія повністю матеріалізує кожен етап у list; lazy-версія – це `build_pipeline`. Обидві використовують ту саму функцію перевірки `parse_student` і дають однаковий Summary (перевіряється assert). Час – найкращий із 2–3 запусків без tracemalloc, пам'ять – пік tracemalloc в окремому запуску, sink – `summarize`.

```
def eager_process(path: Path) -> list[StudentRecord]:
    lines = path.read_text(encoding="utf-8-sig").splitlines()
    cleaned = [line.strip() for line in lines if line.strip() and not line.startswith("#")]
    rows = list(csv.DictReader(cleaned))
    parsed = [parse_student(row) for row in rows]
    students = [item for item in parsed if isinstance(item, Student)]
    return [StudentRecord(*s, level=grade_level(s.average_grade)) for s in students]
```

Таблиця 3 – Elapsed time повної обробки

Records	Eager time, с	Lazy time, с	Lazy / eager
10 000	0.032	0.037	1.14
100 000	0.380	0.356	0.94
500 000	2.286	1.808	0.79

Таблиця 4 – Peak memory і time to first result

Records	Eager peak, МБ	Lazy peak, МБ	Eager / lazy	1-й результат eager, с	1-й результат lazy, мс
10 000	8.3	0.07	111	0.031	0.149
100 000	82.6	0.07	1 109	0.359	0.189
500 000	415.1	0.07	5 563	2.108	0.155

Таблиця 5 – Early termination: перший студент групи PI-25 з балом ≥ 99.97 (1 000 000 записів)

Варіант	Час	Peak memory
eager (повний filtered list, потім [0])	4.479 с	831.3 МБ
lazy (`find_first`)	3.380 мс	75.3 КБ

Аналіз. Пікова пам'ять eager росте лінійно з кількістю записів (8.3 → 415.1 МБ), а lazy залишається ≈0.07 МБ (буфер читання, поточний запис і лічильники) – у 5 563 рази менше на 500 000 записів. За загальним часом на 10 000 записів lazy повільніший (lazy/eager = 1.14), на 500 000 записів – швидший (lazy/eager = 0.79): обидві версії виконують ту саму обчислювальну роботу, тож різниця в часі невелика; найімовірніше, її дають виділення й звільнення пам'яті під проміжні списки в eager-версії. Тому висновок

«generator завжди швидший» неправильний: його головні переваги – пам'ять і перший результат, який lazy-конвеєр дає за 0.16 мс проти 2.11 с. Early termination на 1 000 000 записах знаходить збіг на позиції 651, прочитавши лише початок файла, – у 1 325 разів швидше і з 75 КБ пам'яті замість 831 МБ.

Недоліки lazy-підходу: конвеєр one-shot (для другого проходу його треба створити заново), немає `len()` та індексації, помилки даних виникають під час споживання, а файл залишається відкритим, доки генератор не завершено. Eager доцільний для невеликих наборів із повторним доступом і сортуванням; lazy – для великих файлів, потоків і пошуку з ранньою зупинкою.

9. Перевірка правильності

Тести (`tests/`, unittest) перевіряють протокол iterator, BOM, рекурсивний пошук файлів, кожну причину відхилення та граничні значення балу, лінивість генератора, early termination, топ-N із рівними балами, пакети, eager = lazy, узгодженість згенерованих даних, експорт і те, що lazy займає щонайменше в 10 разів менше пам'яті. Пакет встановлюється командою `pip install -e .` у чисте venv.

Таблиця 6 – Результати перевірок

Перевірка	**Результат**
`python -m unittest discover -s tests`	Ran 33 tests in 1.141s — OK
`ruff check src tests`	All checks passed!
`ruff format --check src tests`	14 files already formatted
`mypy src tests`	Success: no issues found in 14 source files

Висновки

Побудовано потоковий конвеєр обробки журналу студентів (варіант 1), у якому кожен запис проходить етапи читання, очищення, parsing, validation, filtering, transformation і aggregation лише тоді, коли його запитує споживач. Використано власний iterator, окремий iterable, generator functions, yield from (зокрема рекурсивний), generator expressions і шість засобів itertools, а також `heapq.nlargest` для топ-N. Експеримент показав, що lazy-обробка тримає пам'ять сталою і дає перший результат майже миттєво, тоді як eager потребує пам'яті, пропорційної розміру файла; за загальним часом різниця невелика й залежить від обсягу даних. Тож вибір між eager і lazy залежить від обсягу даних і способу їх використання.